

Microservice_Architecture_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
2. Service Decomposition Principles
 - 2.1 Domain-Oriented Decomposition (DDD Bounded Context)
 - 2.2 Decomposition Priority
3. Technology Stack
4. Service Communication
 - 4.1 Communication Pattern Selection
 - 4.2 API Conventions
 - 4.3 Timeout & Retry
5. Service Governance
 - 5.1 Registration & Discovery
 - 5.2 Rate Limiting & Degradation
 - 5.3 Configuration Management
6. Transaction Consistency
 - 6.1 Cross-service Transaction Strategy
 - 6.2 Saga Orchestration Example (Underwriting)
7. Kubernetes Deployment
8. Version Management

Microservice Architecture Standard

1. Overview

This document defines the microservice architecture standards for the insurance technology platform, including service decomposition principles, technology selection, service governance, and communication patterns. It references best practices from Spring Cloud / Kubernetes ecosystem and large enterprise experience.

2. Service Decomposition Principles

2.1 Domain-Oriented Decomposition (DDD Bounded Context)

Domain	Bounded Context	Candidate Service
Product	Product definition, rate, clause	product-service
Underwriting	Application, risk assessment, issuance	underwriting-service, risk-engine
Policy	Policy lifecycle	policy-service
Servicing	Endorsement, reinstatement, cancellation	servicing-service
Claims	FNOL, investigation, assessment, calculation	claim-service, assessment-service
Renewal	Premium payment, grace period, lapse	renewal-service
Reinsurance	Ceded/assumed, contract, billing	reinsurance-service
Channel	Channel management, commission, agreement	channel-service, commission-service
Payment	Collection, payment, reconciliation	payment-service, reconciliation-service
Customer	Customer info, authentication	customer-service
User	Employee, agent accounts	user-service

2.2 Decomposition Priority

1. **Business Responsibility** — functionality within same bounded context
2. **Change Frequency** — separate high-frequency from low-frequency changes
3. **Data Independence** — each service owns its database (Database per Service)
4. **Team Structure** — inverse Conway maneuver, align with team organization
5. **Performance Requirements** — high-throughput services scale independently

3. Technology Stack

Component	Technology	Notes
Framework	Spring Boot 3.x / Spring Cloud 2023	Java 17+
Registry	Nacos / Consul	AP + CP mode support
Config Center	Nacos Config / Apollo	Hot-reload support
Gateway	Spring Cloud Gateway / Kong	Routing, auth, rate limiting
RPC	OpenFeign + Resilience4j	Declarative calls + circuit breaker
Message Queue	RocketMQ / Kafka	Async decoupling, eventual consistency
Container	Kubernetes + Istio	Service mesh
Observability	OpenTelemetry + Prometheus + Grafana	Three pillars

4. Service Communication

4.1 Communication Pattern Selection

Scenario	Pattern	Protocol
Sync Query	RESTful API / gRPC	HTTP/2, Protobuf
Command Write	Async message + Event-driven	RocketMQ / Kafka
Cross-domain Transaction	Saga (orchestration)	Message + local transaction table
State Change Notification	CloudEvents / Domain Events	Message queue
File Transfer	Object storage + message notification	S3/MinIO + MQ

4.2 API Conventions

- URL Pattern: `/ {version} / {domain} / {resource}`
- Versioning: URL Path Versioning (`/v1/policies`)
- Response: Unified JSON structure
- Error Codes: Domain(2 digits) + Business(4 digits), e.g. 010001

4.3 Timeout & Retry

Type	Timeout	Retries	Circuit Breaker
Sync Query	3s	1	50% failure / 10s
Sync Write	5s	0	30% failure / 10s
Async Message	7-day retry	16 (exponential backoff)	-

5. Service Governance

5.1 Registration & Discovery

- All services register with Nacos, health check every 5s
- Weighted load balancing for critical services
- Canary release via metadata tag routing

5.2 Rate Limiting & Degradation

- Gateway: Global rate limiting (token bucket)
- Service: API-level rate limiting (Sentinel / Resilience4j)
- Degradation: Return cached fallback data or friendly error
- Recovery: Half-open state, attempt after 5s

5.3 Configuration Management

- Environment isolation: dev / test / staging / prod
- Configuration changes: Nacos Watch + Apollo gray release
- Sensitive config: Key management service integration, DB encryption

6. Transaction Consistency

6.1 Cross-service Transaction Strategy

- **Strong consistency** (rare): Avoid when possible, local transactions only
- **Eventual consistency** (default): Saga + local message table + scheduled reconciliation
- **Compensation**: Each cross-domain write has a corresponding compensation/rollback interface

6.2 Saga Orchestration Example (Underwriting)

Application → Payment → Policy Issue → Commission Calculation

Compensation: Payment failure → Cancel application; Issue failure → Initiate refund Saga

7. Kubernetes Deployment

Config	Standard
Requests	CPU: 500m, Memory: 1Gi
Limits	CPU: 2, Memory: 4Gi
Readiness	HTTP GET /actuator/health/readiness, interval 10s
Liveness	HTTP GET /actuator/health/liveness, interval 30s
HPA	CPU > 70% or Memory > 80% triggers scale
PDB	minAvailable: 1 (≥ 3 replicas) or 50%

8. Version Management

- Semantic versioning: MAJOR.MINOR.PATCH
- Image tag: {service-name}:{semver}-build.{build-number}
- API version: MAJOR bump for breaking changes, MINOR for additive changes